

Project Report

Lexical Analysis (Method #1: Manual DFA)

1. Compilation Environment and Method

This project was implemented within the Docker environment specified for the course, running on a Linux subsystem (WSL). This ensures the grading environment is accurately replicated.

Environment	WSL/Linux using the provided Docker container. docker run --name compiler2025 --rm -it -v "\$PWD":/work -w /work cs-compiler-hw:1.0
Tool	GCC (GNU Compiler Collection).
Method	Manual Deterministic Finite Automaton (DFA) coded in C.
Core Files Modified	globals.h, util.c, scan.c
Target Executable	cminus_cimpl
Compilation Command	make cminus_cimpl
Execution Command	./cminus_cimpl <testfile>.txt > <output>.txt or ./cminus_cimpl <testfile>.txt
Check Results Command	diff -y <result>.txt <output>.txt

```
root@21e49700deab:/work# make cminus_cimpl
make: 'cminus_cimpl' is up to date.
root@21e49700deab:/work# ./cminus_cimpl test.1.txt

C-MINUS COMPILATION: test.1.txt
4: reserved word: int
4: ID, name= gcd
4: (
4: reserved word: int
4: ID, name= u
4: ,
4: reserved word: int
4: ID, name= v
4: )
5: {
6: reserved word: if
6: (
6: ID, name= v
```

```
root@21e49700deab:/work# ./cminus_cimpl test.1.txt > output.1.txt
root@21e49700deab:/work# diff -y result.1.txt output.1.txt

C-MINUS COMPILATION: ./test.1.txt | C-MINUS COMPILATION: test.1.txt
4: reserved word: int              4: reserved word: int
4: ID, name= gcd                   4: ID, name= gcd
4: (                               4: (
4: reserved word: int              4: reserved word: int
4: ID, name= u                     4: ID, name= u
4: ,                               4: ,
4: reserved word: int              4: reserved word: int
4: ID, name= v                     4: ID, name= v
4: )                               4: )
5: {                               5: {
6: reserved word: if               6: reserved word: if
6: (                               6: (
6: ID, name= v                     6: ID, name= v
6: ==                             6: ==
6: NUM, val= 0                     6: NUM, val= 0
6: )                               6: )
```

2. Implementation and Operation

The C-Minus scanner was created by implementing a state-based DFA within the C function `getToken()` found in `scan.c`. The implementation required modifications across three main files:

globals.h

The enum `TokenType` was expanded to include all C-Minus reserved words (e.g., `INT`, `VOID`, `WHILE`) and the 19 required special symbols (e.g., `NE`, `LE`, `GE`, `LCURLY`, `RBRACE`).

util.c

`reservedLookup()`: The static lookup table within this function was expanded to map the six C-Minus keywords (`if`, `else`, `while`, `return`, `int`, `void`) to their corresponding `TokenType` values. Any other string is classified as an ID.

`printToken()`: The switch-case structure was updated to correctly print the lexeme for all new single and double-character special symbols (e.g., printing `==` for the `EQ` token).

scan.c

The `getToken()` function implements the DFA using `StateType` enums and a central switch-case loop.

To handle multi-character tokens and comments, new states were defined beyond the basic TINY set:

The DFA is modeled using the enum `StateType`, which defines the scanner's states.

Defined States

- `START`: Initial state, determines what type of token begins.
- `IN_NUM`: Reading a sequence of digits (numbers).
- `IN_ID`: Reading an identifier or reserved word.
- `IN_DIV`: After reading `'/'`, decides whether it's division (`/`) or the start of a comment (`/*`).
- `IN_COMMENT`: Inside a block comment `/* ... */`.
- `IN_CMT_END`: After reading `*` inside a comment, waiting for `/` to close it.
- `IN_EQ_OR_ASSIGN`: After `=`, determines if it's `=` (`ASSIGN`) or `==` (`EQ`).
- `IN_NE`: After `!`, checks if it's `!=` or an error.
- `IN_LT_OR_LE`: After `<`, determines `<` (`LT`) or `<=` (`LE`).
- `IN_GT_OR_GE`: After `>`, determines `>` (`GT`) or `>=` (`GE`).
- `DONE`: A token has been fully recognized.

Key Logic

- *Identifiers and Numbers*

- If the first char is a digit, go to IN_NUM until a non-digit is found → token NUM.
- If the first char is a letter, go to IN_ID until a non-alphanumeric is found → token ID or reserved keyword (via reservedLookup()).

- **Comments**

Handled with the sequence:

START → IN_DIV → IN_COMMENT → IN_CMT_END → START

When ‘ / ’ is read: If followed by ‘ * ’, it becomes a comment. The ‘ / ’ is removed from the Token buffer (tokenStringIndex--).

Otherwise, it's the division operator (OVER).

- In IN_COMMENT:

All characters are ignored (save = FALSE).

If EOF occurs inside → token ERROR.

If ‘ * ’ is found → move to IN_CMT_END.

- In IN_CMT_END:

If followed by ‘ / ’ → comment closes, back to START.

If not → return to IN_COMMENT.

This ensures that no characters from comments are ever stored in a token.

- **Operators with Lookahead**

The DFA uses lookahead + controlled backtracking (ungetNextChar()) for multi-character operators:

= → ASSIGN or EQ (==)

! → error or NE (!=)

< → LT or LE (<=)

> → GT or GE (>=)

If the second character doesn't match, the scanner backtracks and finalizes the single-character token.

Lexical Analyzer Implementation (Method 2: Flex/Lex)

1. Compilation Environment and Method

Environment	WSL/Linux using the provided Docker container.
Tool	Flex (Lexical Analyzer Generator) and GCC (C Compiler).
Method	Specification of C-Minus lexical patterns using Regular Expressions in the cminus.l file.
Core Files Modified	cminus.l (Lexical rules). lex.yy.c (Auto-generated C file by Flex).
Target Executable	cminus_lex
Compilation Command	flex cminus.l gcc main.c util.c lex.yy.c -o cminus_lex -lfl
Execution Command	./cminus_lex <test.txt> > <output.lex.txt> or ./cminus_lex <test.txt>
Check Results Command	diff -y <result>.txt <output.lex>.txt

Method 2 implements the C-Minus lexical analyzer using the Flex tool (Fast Lexical Analyzer Generator). This approach simplifies the scanner implementation by defining lexical patterns using regular expressions in the cminus.l file, replacing the complex manual Deterministic Finite Automaton (DFA) logic required in Method 1.

```

root@21e49700deab:/work# flex cminus.l
root@21e49700deab:/work# gcc main.c util.c lex.yy.c -o cminus_lex -lfl
main.c:48:1: warning: return type defaults to 'int' [-Wimplicit-int]
  48 | main( int argc, char * argv[] )
      | ^~~~~
util.c:119:8: warning: type defaults to 'int' in declaration of 'indentno' [-Wimplicit-int]
  119 | static indentno = 0;
      |          ^~~~~~
root@21e49700deab:/work# ./cminus_lex test.1.txt

```

```

C-MINUS COMPILATION: test.1.txt
4: reserved word: int
4: ID, name= gcd
4: (
4: reserved word: int
4: ID, name= u
4: ,
4: reserved word: int
4: ID, name= v
4: )
5: {
6: reserved word: if

```

```

root@21e49700deab:/work# ./cminus_lex test.1.txt > output.1.lex.txt
root@21e49700deab:/work# diff -y result.1.txt output.1.lex.txt

```

```

C-MINUS COMPILATION: ./test.1.txt | C-MINUS COMPILATION: test.1.txt
4: reserved word: int 4: reserved word: int
4: ID, name= gcd 4: ID, name= gcd
4: ( 4: (
4: reserved word: int 4: reserved word: int
4: ID, name= u 4: ID, name= u
4: , 4: ,
4: reserved word: int 4: reserved word: int
4: ID, name= v 4: ID, name= v
4: ) 4: )
5: { 5: {
6: reserved word: if 6: reserved word: if

```

2. Modified Lexical Specification: cminus.l

The core of Method 2 was modifying the provided TINY specification (cminus.l) to meet the C-Minus lexical conventions. This involved updating keywords, multi-character symbols, and critically, the comment handling.

Key Changes

Keywords

Replaced TINY's keywords (then, end, read, etc.) with C-Minus reserved words: if, return, else, int, while, void. Ensured compliance with the C-Minus language specification.

Special Symbols

Added rules for multi-character C-Minus relational operators: ==, !=, <=, >=. Defined tokens for symbols not present in the original TINY grammar.

Comment State:

Defined the exclusive start condition %x COMMENT. Dedicated state required for handling multi-line C-style comments (/* ... */).

- `"/*` BEGIN(COMMENT); Switches the scanner to the COMMENT state.
- `<COMMENT>"*/"` BEGIN(INITIAL); Returns the scanner to the default token recognition state.
- `<COMMENT>\n` lineno++; Correctly increments the global line counter for multi-line comments.
- `<COMMENT>.` { /* Ignore */ } Matches and skips every character inside the comment that is not a newline or the end delimiter.
- `<COMMENT><<EOF>>` Reports an "unclosed comment" error if the file ends prematurely within the comment state.